Semantic representations : word embeddings and named entity recognition

Lab report, NLP Today course

October 2026

Group members:

- Mouhamed Nourou Dine CISSE
  mouhamed-nourou-dine.cisse@universite-paris-saclay.fr
- Mouhamadou Moustapha DIOP
  mouhamadou-moustapha.diop@universite-paris-saclay.fr

1 Introduction

This report presents our work for the lab sessions of the Semantic Representations part of the course. The first lab consists in building and comparing static word embeddings, that is representations where every occurrence of a word gets the same vector whatever its context. We use three approaches: word2vec CBOW, word2vec skip-gram and fastText CBOW. Each approach is trained on two French corpora that differ in size and in domain, a small medical corpus (QUAERO_FrenchMed) and a larger news corpus (QUAERO_FrenchPress), which gives six models.

Following the instructions, we compare these models through semantic similarity: for five candidate words (*patient*, *traitement*, *maladie*, *solution* and *jaune*), we look at the ten closest words in terms of cosine similarity. This evaluation relies on the idea stated in the course: if two word vectors are close, the words should be semantically or syntactically close. In the second lab, these embeddings are used to train LSTM and CNN models for named entity recognition, which are compared with a BERT model fine-tuned on the same task.

2 Lab 1: word embeddings training

2.1 Training setup

Corpora. Both files are already tokenized, with one sentence per line and tokens separated by spaces. Our preprocessing is lowercasing, so that forms such as *Traitement* and *traitement* are counted as the same word, which matters when the corpus is small. Table 1 gives the main figures.

Corpus	Sentences	Tokens	Vocabulary
QUAERO_FrenchMed	3,021	51,896	8,088
QUAERO_FrenchPress	38,548	1,251,561	38,279

Table 1: Size of the two training corpora after lowercasing

The news corpus contains about 24 times more tokens than the medical one.

Approaches. The three approaches learn the embeddings while solving a word prediction task:

- CBOW predicts the current word from the average of the embeddings of its context words;
- skip-gram does the opposite: from the current word, it predicts each word of its context;
- fastText represents a word by the average of its own vector and of the vectors of its character n-grams. We use its CBOW version, as asked in the instructions. Thanks to the n-grams, fastText can also build a vector for a word that never appears in the training data.

Hyperparameters. As recommended in the instructions, we use vectors of dimension 100 and min_count = 1, so that every word of the corpus gets a vector. The context window is 5 (up to five words on each side of the current word) and the other parameters keep their gensim default values (5 training epochs, negative sampling with 5 negative words). The six models are trained with the `Word2Vec` and `FastText` classes of the gensim library. The results of this section are obtained with these 5 epochs; for the second lab, the models were retrained with 30 epochs (Section 3).

To make the results reproducible, we fixed the random seed (seed = 42) and trained on a single thread (workers = 1). With several threads, the order of the updates changes from one run to another, and the nearest neighbours change slightly even with a fixed seed. Finally, the vectors of the six models are saved in the word2vec binary format, to be reused in the second lab.

2.2 Semantic similarity: same corpus, different approaches

To measure the impact of the approach, we compare the three models trained on the medical corpus. Table 2 gives the first five neighbours of each candidate word (the full lists of ten neighbours are in the notebook) and, in the last row, the range of the cosine similarities of the ten neighbours.

Word	CBOW	Skip-gram	fastText
patient	par, en, avec, pendant, dans	qui, symptômes, recommandé, devra, nécessaire	patiente, conscients, ingrédients, présentaient, pansements
traitement	le, par, des, sur, dans	tysabri, le, médicaments, tasmar, ayant	taaitement, allaitement, évitement, traitements, étroitement
maladie	en, des, la, avec, du	association, la, acide, administration, infection	malade, prolactine, sociale, héparine, de
solution	contient, flacon, (, mg,)	flacon, contient, 100, mg, jour	dissolution, évolution, evolution, dilution, pollution
jaune	ou, avec, pour, ces, résultats	13, puis, 24, boîtes, ±	une, immune, minerale, intestinale, anale
Cosine range	0.998 to 1.000	0.945 to 0.998	above 0.999

Table 2: First five nearest neighbours of the candidate words for the three models trained on the medical corpus

CBOW. The CBOW model does not produce usable neighbours. For four words out of five, the closest words are function words (*le*, *par*, *des*, *en*) or punctuation, and all cosine similarities lie between 0.998 and 1.000. When every neighbour gets almost the same score, the ranking no longer means anything: the candidate words and their neighbours have almost the same direction, and the most frequent words come first because they appear in every context. Only *solution* gets related words (*contient*, *flacon*, *mg*), because it appears very often in the same fixed expressions of drug leaflets (*un ml de solution contient ...*).

We explain this failure by the small size of the corpus combined with the way CBOW learns. In CBOW, each position of the text gives a single prediction, made from the merged context, whereas skip-gram turns every occurrence of a word into several training examples, one per context word. With about 52,000 tokens and 5 epochs, CBOW does not receive enough training signal to separate the words. This agrees with the authors of word2vec, who present skip-gram as better suited to infrequent words.

Skip-gram. With skip-gram, the neighbours start to make sense in the medical domain. *traitement* is close to *médicaments* and to drug names (*tysabri*, *tasmar*, and *epivir* further in the list), that is, to actual treatments. *solution* is close to *flacon*, *mg* and *injectable*: the model has learned the pharmaceutical meaning of the word (a liquid form of a drug, as in *solution pour perfusion*), not its everyday meaning. *maladie* gets *infection* and *cancer* among its ten neighbours, and *patient* gets *symptômes* and *administré*. The scores are also lower and more spread (0.945 to 0.998), which makes the ranking more informative.

Two limits remain. Some function words are still present (*le*, *qui*, *la*), and *jaune*, which occurs only 9 times, gets numbers and packaging words (*boîtes*, and *titane* further in the list). The corpus explains these neighbours: *jaune* mostly appears in descriptions of tablets and in lists of excipients (*oxyde de fer jaune*, *dioxyde de titane*). The model therefore reflects the use of the word in this corpus, but 9 occurrences are not enough to get a reliable representation.

fastText. The fastText neighbours are of a different nature: they share long character sequences with the candidate word. *patient* is close to *patiente*, *maladie* to *malade*, and *traitement* to *traitements* and even to the misspelled forms *taaitement* and *traitment*, which occur in the corpus. This is the direct effect of the character n-grams: two words that share many n-grams get close vectors. This property is useful, since it groups inflected forms and spelling errors and gives vectors to unseen words, which should help for named entity recognition in the second lab.

However, the similarity then follows the form more than the meaning. *allaitement*, *évitement* and *étroitement* are close to *traitement* only because of the ending *-ement*, *solution* is close to *évolution* and *pollution* because of *-lution*, and *jaune* to *une* and *immune*. Like CBOW, whose architecture it uses here, fastText gives scores above 0.999, which suggests that on such a small corpus the shared n-grams weigh more than the contexts in the final vectors.

2.3 Semantic similarity: same approach, different corpora

We now compare each approach across the two corpora to measure the impact of the data. Two factors change at the same time: the quantity of data (24 times more tokens in the news corpus) and its type (news instead of medical texts). Table 3 gives the first five neighbours for the models trained on the news corpus, together with the number of occurrences of each candidate word in both corpora.

Word	Occurrences	CBOW	Skip-gram	fastText
patient	34 / 11	artificiel, morceau, délit, constituer, éthique	motivation, accumulé, identification, brevet, accuser	patientent, impatient, trient, détient, coefficient
traitement	288 / 53	contenu, trésor, collectif, fréjus, nucléaire	mépris, trésor, potentiel, coût, fonctionnement	concrètement, comportement, farouchement, promptement, fondement
maladie	72 / 114	reprise, somme, douleur, bataille, négociation	prime, drogue, décote, pauvreté, nationalité	malnutrie, trilogie, magie, ossétie, démographie
solution	70 / 100	seule, difficulté, querelle, garantie, suffisante	légitimité, pacifique, consensuelle, difficulté, crédibilité	révolution, résolution, évolution, pollution, dissolution
jaune	9 / 23	michael, baba, peter, maréchal, lars	maillot, cooke, emparé, sprint, baden	abdoune, lune, brune, saadoune, saâdoune
Cosine range		0.855 to 0.949	0.814 to 0.954	0.921 to 0.992

Table 3: First five nearest neighbours of the candidate words for the three models trained on the news corpus. The column (Occurrences) gives the number of occurrences of each word in the medical and in the news corpus

Quantity of data. With 24 times more tokens, the similarity scores are no longer saturated: they range from 0.855 to 0.949 for CBOW and from 0.814 to 0.954 for skip-gram, against 0.998 to 1.000 and 0.945 to 0.998 on the medical corpus. The vectors are therefore better spread in the space, and the ranking of the neighbours becomes meaningful. The most visible change concerns CBOW: its neighbours are no longer function words but content words (*difficulté* and *garantie* for *solution*, *douleur* for *maladie*). The failure observed in the previous section was therefore mainly due to the lack of data. For fastText, on the contrary, the larger corpus does not change the nature of the neighbours, which still share the ending of the candidate word.

Type of data. More data does not mean better neighbours for our candidate words, because the news corpus does not use them in the same way. Most of them are medical words, and the news corpus uses them either less often (*patient* appears 11 times against 34, *traitement* 53 times against 288) or with another meaning. Looking at the corpus explains the neighbours given by skip-gram :

- *solution* is used in its everyday meaning (*trouver une solution*, *une solution de compromis*), hence neighbours such as *consensuelle*, *pacifique* or *alternative*, far from the pharmaceutical meaning learned on the medical corpus;
- *traitement* appears in social and economic contexts (*le traitement du chômage*, *la différence de traitement entre les salariés*), hence *coût*, *financement* or *financier*;
- *maladie* often appears in social debates (*assurance maladie* occurs 19 times), hence *prime*, *décote* or *pauvreté*;
- *jaune* is mostly used in sports news: 14 of its 23 occurrences are in *maillot jaune*, which explains *maillot* and *sprint*;
- *patient*, with only 11 occurrences, some of them as an adjective (*être patient*), gets neighbours without a clear link to it.

3 Lab 2: named entity recognition

The second lab evaluates the six embeddings of Lab 1 on named entity recognition (NER), where a label must be predicted for every word: B- for the first word of an entity, I- for the following ones, and O outside entities. We compare two families of models: LSTM and CNN models whose input layer is initialized with our static embeddings, and a BERT model fine-tuned on the task. Following the instructions, all models are evaluated in terms of precision, recall and F1 on the test set, computed with seqeval at the entity level: an entity is counted as correct only if both its boundaries and its type are right. We do not report the accuracy, which is computed word by word and dominated by the O label (88% of the words of the news training corpus).

We use two annotated corpora, described in Table 4. The medical corpus QUAERO MEDLINE contains titles of scientific articles, with 10 entity types: anatomy, chemicals and drugs, devices, disorders, geographic areas, living beings, objects, phenomena, physiology and procedures. The news corpus QUAERO Press is much larger and has 5 entity types: persons, locations, organisations, functions and products.

	MEDLINE	News
Sentences (training / validation / test)	834 / 832 / 835	35,723 / 2,825 / 2,880
Entity types	10	5
Entities in the training set	2,198	70,917
Words labelled O in the training set	68.6%	88.3%
Average sentence length (training set)	13.8 words	32.4 words

Table 4: The two NER corpora

Before this lab, we retrained the six embeddings of Lab 1 with 30 epochs instead of the default 5, all the other settings being unchanged. Lab 1 showed that the medical CBOW model was undertrained, and more epochs fix this: its cosine similarities now range from 0.779 to 0.978 instead of 0.998 to 1.000, and its neighbours become meaningful (*solution* is close to *poudre*, *injectable* and *buvable*).

3.1 LSTM and CNN models

Adaptation of the provided script. The script given for this lab (`cnn_classification.py`) classifies whole sentences: it reads one label per sentence and only keeps the last output of the network. For NER we need one prediction per word, so we adapted it (`scripts/ner_lstm_cnn.py`). The files are read word by word (the word is the second column and the label the last one), the network outputs a score for every label at every position, and the loss is the cross-entropy computed on all the words of the sentence. Sentences are grouped in batches of 32 and padded to the length of the longest sentence of the batch; the padding positions are ignored by the loss and by the evaluation.

Preprocessing. Words are lowercased, since the embeddings of Lab 1 were trained on lowercased text. This loses the capital letters, which are a useful clue for named entities, but it is the only way to find the words in our embeddings. Labels are put in uppercase: the labels of the news corpus are written in lowercase (*b-pers*), and seqeval does not recognize the prefix *b-* as the beginning of an entity, so two consecutive entities of the same type would be merged into one. The vocabulary contains the words of the training, validation and test files, so that a test word never seen during training can still get its vector from Lab 1.

Architectures. Figure 1 shows the two models. Both start with an embedding layer of dimension 100 and end with a linear layer that predicts the label of each word, with a dropout of

0.5 before it. The LSTM is bidirectional, with 128 units in each direction, and we use packed sequences so that it does not read the padding. The CNN has two convolutions with windows of 3 and 5 words and 100 filters each, followed by a ReLU, and their outputs are concatenated. The main difference between the two models is the context they use: the LSTM can use the whole sentence to predict the label of a word, while the CNN only sees at most 5 words around it.

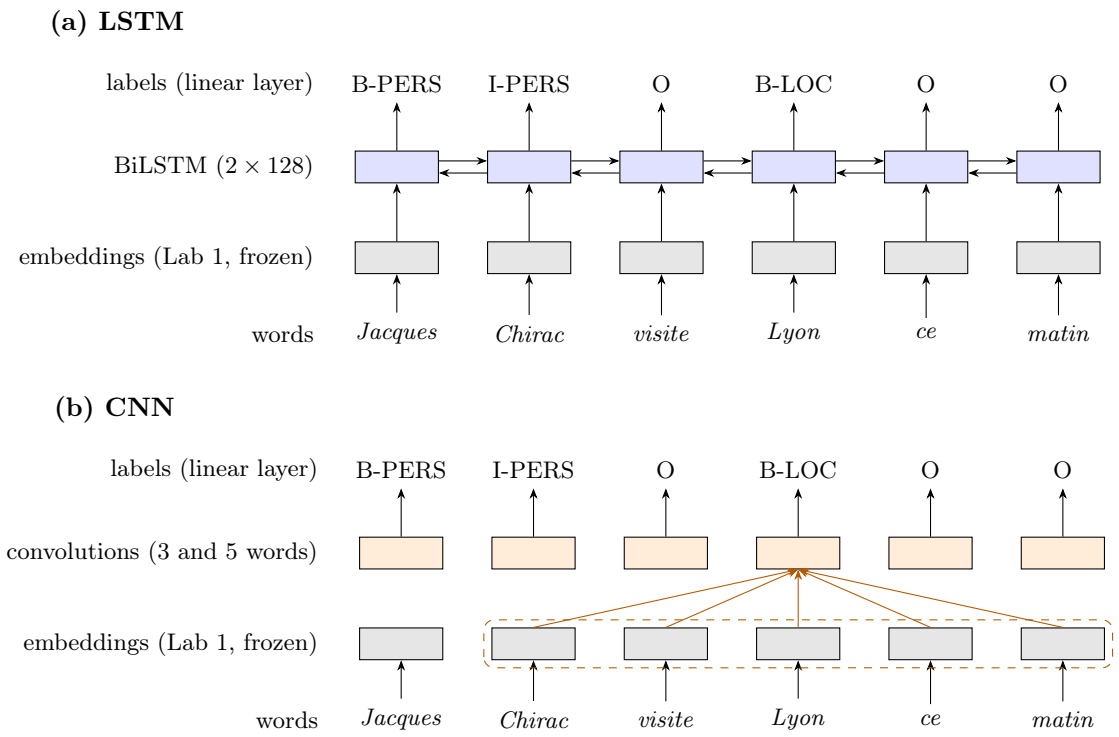


Figure 1: The two models on an example sentence. (a) The bidirectional LSTM passes information along the sentence in both directions, so each prediction can use the whole sentence. (b) The CNN computes the representation of a word from a window of 3 and a window of 5 words centred on it (shown here for *Lyon* with the window of 5).

Embeddings. The embedding layer is initialized with one of the six embeddings of Lab 1, and the words without a vector get a random one. As a reference, we also train each model with random vectors for all the words. The embeddings are kept frozen during training. We made this choice after a first test with the LSTM on MEDLINE: when the embeddings are updated, the medical skip-gram vectors and random vectors give the same F1 (0.318 and 0.319), because the network relearns its own vectors from the training data; when they are frozen, the skip-gram vectors clearly beat random ones (0.261 against 0.176). Freezing is therefore what allows us to measure the information carried by the embeddings themselves.

Training. We use the Adam optimizer with a learning rate of 0.001 and a fixed seed. The LSTM is trained for at most 50 epochs on MEDLINE and 40 on the news corpus, and the CNN, which converges more slowly, for at most 150 and 80 epochs. After each epoch we compute the F1 on the validation set and keep the model of the best epoch, which is then evaluated once on the test set. This gives 28 configurations: 2 models, 7 embeddings (the six of Lab 1 and the random reference) and 2 corpora.

3.2 BERT

In order to benchmark our LSTM and CNN approaches based on static word embeddings trained during lab 1, we fine-tuned pretrained Transformer-based language models (CamemBERT) for the

NER task. The objective was to evaluate whether contextualized representations could improve NER performance compared with the previously tested architectures.

We used Hugging Face's `Transformers` library and fine-tuned the models using AutoModelForTokenClassification. For the medical corpus, we used `almanach/camembert-bio-base`, a CamemBERT model pretrained on biomedical data. For the French Press corpus, we used `camembert-base`. Both models were trained separately on their respective training sets.

3.2.1 Dataset preparation and tokenization

The original datasets (.conll files) are provided as sequences of words associated with BIO entity labels where different sentences are seperated by blank lines. We first converted each sentence into two aligned lists: a list of tokens and a list of corresponding labels. We made sure that the number of tokens and labels was identical for every sentence. We transformed the list of tokens and labels into a huggingface Dataset object and computed the id2label and label2id dictionnaries from he training datas.

The pretrained tokenizer was then applied with the option `is_split_into_words=True`, which allows the original word-level annotations to be aligned with the subword tokens produced by the Transformer tokenizer.

Since Transformer tokenizers can split a single word into several subtokens, the original word-level label was assigned only to the first subtoken. Subsequent subtokens were assigned the value `-100`. Special tokens were also assigned `-100`. These positions are ignored by the token-classification loss during training.

The alignment procedure can be summarized as follows:

$$\text{Word} \rightarrow \text{Subword}_1, \text{Subword}_2, \ldots, \text{Subword}_n$$

$$\text{Label} \rightarrow \text{Label}, -100, \ldots, -100$$

The label mapping was created from the training set and then reused for the validation and test sets to ensure consistent label indices across all splits.

3.2.2 Model architecture

We used `AutoModelForTokenClassification`, which adds a token-level classification head on top of the pretrained Transformer.

For an input sequence, the Transformer produces a contextual representation for every input token:

$$H = (h_1, h_2, \ldots, h_T)$$

where T is the number of input tokens and h_i is the contextual representation of token i.

A classification layer then maps each representation to the set of NER labels:

$$z_i = W h_i + b$$

where W and b are the parameters of the token-classification head. The predicted label probabilities are obtained using the softmax function:

$$P(y_i = k \mid x) = \frac{\exp(z_{i,k})}{\sum_{j=1}^{K} \exp(z_{i,j})}$$

where K is the number of NER labels.

The model is fine-tuned end-to-end, meaning that the pretrained Transformer parameters and the newly initialized classification head are updated during training.

3.2.3 Training

We used the Hugging Face `Trainer` API to perform the training procedure. The training configuration used a batch size of 8, a learning rate of 10^{-5}, a weight decay of 0.01, and evaluation after every epoch. The medical model was trained for 15 epochs, while the French Press model was trained for 5 epochs.

The training procedure can be represented as:

Input tokens → Tokenizer → CamemBERT → Token representations → Classification head → NER predictions

For each batch, the model computes the token-level cross-entropy loss while ignoring positions associated with `-100`. The loss is then backpropagated through the Transformer and classification head to update the model parameters.

The `DataCollatorForTokenClassification` was used to dynamically pad batches and construct the corresponding tensors required by the model.

3.2.4 Evaluation

For evaluation, the predictions produced by the model were converted from label IDs back into their corresponding BIO labels. Positions with a ground-truth label of `-100` were removed before computing the metrics.

We used the `seqeval` library to compute precision, recall, and F1-score at the entity level:

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

$$F_1 = 2\frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Unlike simple token-level accuracy, entity-level F1 evaluates whether complete named entities are correctly identified, making it a more appropriate metric for NER.

3.3 Results

Table 5 gives the scores of all the models on the two test sets.

All the scores are much lower on MEDLINE than on the news corpus. Its training set contains 32 times fewer entities, spread over twice as many types (Table 4), which leaves about 220 examples per type against more than 14,000.

For the LSTM and CNN, the precision is higher than the recall in 27 configurations out of 28: these models miss many entities, but the entities they find are more often correct. The only exception, the LSTM with the news fastText embeddings on the news corpus, is discussed below.

On the news corpus, the test F1 of the LSTM and CNN is about 0.1 lower than their validation F1 (for example 0.562 against 0.654 for the LSTM with the news CBOW embeddings), while on MEDLINE the two are close. The news test set contains fewer entities than the validation set (3,539 against 5,176 for about the same number of words), which may partly explain this difference. The ranking of the models is almost the same on both sets.

Figure 2 shows the validation F1 of the LSTM during training. Its curves are almost flat at the end on MEDLINE, and on the news corpus its best epoch is at most 26 out of 40. The CNN learns more slowly, and several CNN models reach their best epoch near the end, but over the last quarter of training their validation F1 increases by less than 0.005, except for the three models trained on the news corpus with the medical embeddings (0.011 to 0.016 over the last 20 epochs). The scores of these three models are therefore slightly underestimated.

		MEDLINE			News		
Model	Embeddings	P	R	F1	P	R	F1
LSTM	random	0.242	0.149	0.184	0.488	0.304	0.375
	CBOW, medical	0.249	0.200	0.222	0.372	0.263	0.308
	skip-gram, medical	0.300	0.227	0.258	0.403	0.277	0.328
	fastText, medical	0.277	0.247	0.261	0.384	0.238	0.294
	CBOW, news	0.291	0.243	0.265	0.651	0.495	**0.562**
	skip-gram, news	0.300	0.269	0.284	0.622	0.500	0.554
	fastText, news	0.303	0.272	0.287	0.431	0.506	0.466
CNN	random	0.265	0.178	0.213	0.584	0.236	0.336
	CBOW, medical	0.273	0.182	0.218	0.442	0.206	0.281
	skip-gram, medical	0.288	0.234	0.258	0.433	0.225	0.296
	fastText, medical	0.283	0.177	0.218	0.425	0.192	0.265
	CBOW, news	0.263	0.218	0.239	0.612	0.425	0.501
	skip-gram, news	0.310	0.270	**0.289**	0.602	0.418	0.494
	fastText, news	0.263	0.205	0.231	0.526	0.436	0.477
BERT	fine-tuned	0.460	0.536	0.495	0.836	0.797	0.816

Table 5: Precision (P), recall (R) and F1 on the test sets. For the LSTM and CNN, the embeddings of Lab 1 are frozen and *random* is the reference with random vectors; the best of these models on each corpus is in bold. BERT is CamemBERT-bio on MEDLINE and CamemBERT on the news corpus.

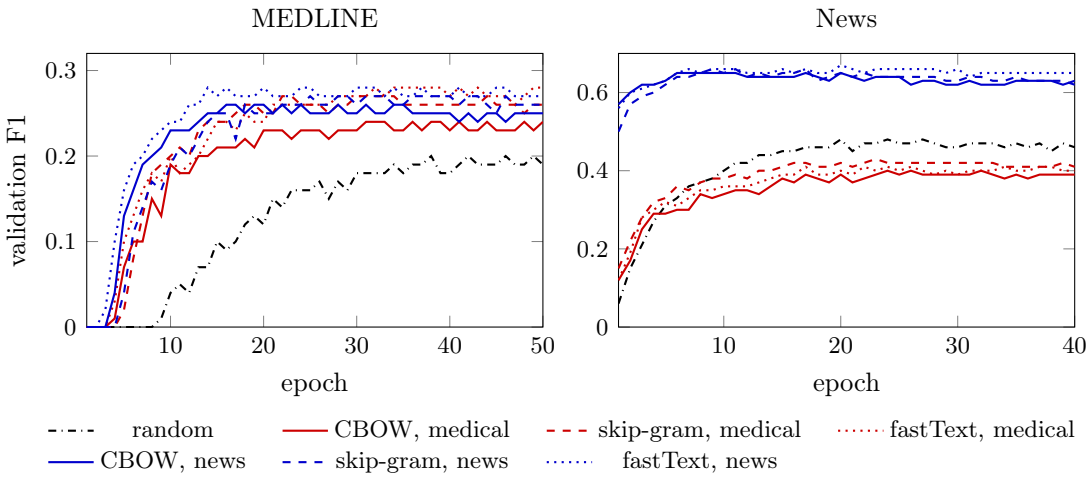


Figure 2: Validation F1 of the LSTM after each epoch for the seven embeddings (medical embeddings in red, news embeddings in blue)

3.4 Answers to the questions

Which model performs best? On both corpora, the fine-tuned BERT model obtains the best scores, with an F1 of 0.495 on MEDLINE and 0.816 on the news corpus; it is compared with the other models in the last question. Among the models based on our static embeddings, the best F1 on MEDLINE is 0.289, obtained by the CNN with the news skip-gram embeddings, almost equal to the LSTM with the news fastText embeddings (0.287). On the news corpus, the best is the LSTM with the news CBOW embeddings (0.562).

Comparing the two architectures, the LSTM is better than the CNN on the news corpus for 6 of the 7 embeddings on the test set, and for all of them on the validation set, although it was trained for fewer epochs (40 against 80). On MEDLINE the two models are close and their ranking depends on the embeddings. This matches the difference of context between the two

models (Figure 1). News sentences are more than twice as long as MEDLINE titles (Table 4), so the words far from the current word can help, whereas in a title of 14 words a window of 5 words already covers a large part of the sentence.

Are there differences between the embeddings of the two corpora? Yes, mainly on the news corpus. To interpret them, Table 6 gives the proportion of the words of each NER corpus that have a vector in our embeddings. The three approaches share the same vocabulary for a given training corpus, so the coverage only depends on the corpus.

	MEDLINE		News	
Embeddings	Words	Occurrences	Words	Occurrences
Medical	77.9	94.8	9.6	67.7
News	41.5	74.2	94.6	99.6

Table 6: Proportion (in %) of the distinct words and of the word occurrences of the NER corpora (training, validation and test files) that have a vector in the embeddings of Lab 1

The covered words are mostly the frequent ones: for instance, the medical embeddings contain only 9.6% of the distinct words of the news corpus, but these words represent 67.7% of its occurrences.

On the news corpus, the news embeddings give by far the best results with both models (F1 from 0.466 to 0.562 with the LSTM), while the medical embeddings give the lowest scores, even below the random reference (0.294 to 0.328 against 0.375 with the LSTM, 0.265 to 0.296 against 0.336 with the CNN). This is not a question of training time. Figure 2 shows that with the LSTM the medical embeddings are even ahead of random vectors during the first epochs, but they stop improving earlier: their best validation F1 (0.403 to 0.427, reached between epochs 22 and 26) stays clearly below the one of random vectors (0.481). A possible explanation is that the vectors of the frequent words were learned on a small corpus of another domain, so they give poor or even misleading information about the news, and since they are frozen the model cannot correct them.

On MEDLINE, the result is less intuitive. The news embeddings give a higher test F1 than the medical ones for each model and each approach (for instance 0.284 against 0.258 for the LSTM with skip-gram), although they cover far fewer words. These differences are small and are not always found on the validation set, but the news embeddings are at least as good as the medical ones on medical texts. They are trained on 24 times more data, so the vectors of the general words (verbs, prepositions, common nouns), which form the context of the entities, are probably more reliable, as Lab 1 already suggested. This agrees with the results presented in the course on spoken language understanding, where embeddings trained on a huge out-of-domain corpus gave better results than embeddings trained on a small task-dependent corpus. The medical embeddings remain useful on MEDLINE: with the LSTM, all of them improve over random vectors (0.222 to 0.261 against 0.184); with the CNN, the gain is clear only for skip-gram.

Across the approaches, skip-gram is the most regular: it gives the best or almost the best embeddings in every setting. The medical CBOW embeddings are among the weakest on MEDLINE (0.222 with the LSTM and 0.218 with the CNN, close to the random reference for the CNN), as in Lab 1, where CBOW was the approach most affected by the small size of the medical corpus. fastText gives mixed results: with the LSTM on the news corpus, its news embeddings obtain the best validation F1 of all the LSTM models (0.668) but only 0.466 on the test set, with a recall higher than the precision. We could not explain this drop, which shows that a single run on one test set is not enough to separate close configurations. Finally, the ability of fastText to build vectors for unseen words is not used here, since we exported only the vectors of the words of the Lab 1 corpora.

Comparison with a Transformer model. The fine-tuned BERT models are far better than all the models based on static embeddings: CamemBERT-bio reaches an F1 of 0.495 on MEDLINE against 0.289 for our best model, and CamemBERT reaches 0.816 on the news corpus against 0.562. The gain comes from both the precision and the recall, but mostly from the recall: compared with our best model on each corpus, it goes from 0.270 to 0.536 on MEDLINE and from 0.495 to 0.797 on the news corpus. BERT therefore finds many entities that the LSTM and CNN miss, and on MEDLINE it is the only model whose recall is higher than its precision.

Several differences explain this gap. BERT gives contextual representations, where the vector of a word depends on the sentence, while our embeddings give the same vector to every occurrence of a word. It is pretrained on a much larger corpus than those of Lab 1 (138 GB of text for CamemBERT, as presented in the course), and CamemBERT-bio is pretrained on biomedical texts, which probably helps with the small size of MEDLINE. Its tokenizer also splits unknown words into subwords and keeps the capital letters, whereas our embeddings do not cover all the words (Table 6) and our preprocessing removes the case. Finally, all its parameters are fine-tuned, but this alone does not explain the gap: in our first test, the LSTM with updated embeddings only reached 0.318 on MEDLINE (Section 3.1). These results agree with the course, where a fine-tuned CamemBERT outperformed a BiLSTM-CNN with CBOW embeddings on the MEDIA task; the gap is much larger here, probably because our static embeddings are trained on small corpora.

4 Conclusion

This first lab shows that static word embeddings depend on both the training approach and the training data. On the small medical corpus and with the default 5 epochs, skip-gram gives the most meaningful neighbours, CBOW does not manage to separate the words (all its similarity scores are close to 1), and fastText mainly captures the form of the words, which groups inflected forms and spelling errors. With 24 times more data, the news corpus gives a better structured space, where CBOW finally works, but the meanings learned are those of the news domain: *solution* becomes the solution to a problem, *traitement* gets a social and economic meaning, and *jaune* is associated with the *maillot jaune*. In short, the quantity of data decides whether a model manages to learn, and the type of data decides what it learns.

This evaluation remains qualitative and relies on only five words, some of which are rare (*jaune* occurs 9 times in the medical corpus), so these conclusions need to be confirmed on a real task.

The second lab confirms these conclusions on named entity recognition. The news embeddings, trained on 24 times more data, give the best LSTM and CNN models on the news corpus and are at least as good as the medical ones on MEDLINE, although they cover far fewer of its words. The medical embeddings improve over random vectors on MEDLINE, but they do not help on the news corpus, where their scores are even below the random reference. Among these models, the LSTM, which reads the whole sentence, is better than the CNN on the long sentences of the news corpus, and all of them remain far below the fine-tuned BERT model.